



Department of Industrial Engineering

System Analysis Final Project

Object-Oriented System Analysis & Design

Report 2 - Process, Collaboration, & Reflection

Professor: Loke Kar Seng

Group Members:

F11301104 Gabriela Caceres

F11301101 Juan Azcona

F11301111 Filippo Maldonado

F11301114 Sofia Sotelo

2026.06.09

Table of Contents

- 1. Team Governance & Timeline**
- 2. Requirements Elicitation Process**
- 3. The CRC Workshop & Roleplay Execution**
- 4. Design Evolution & Decision Log**
- 5. Individual & Team Reflections**

1. Team Governance & Timeline

1.1 Team Governance

The project was completed as a short-term group assignment for Object-Oriented System Analysis & Design. Because the project had to move from requirements elicitation to UML modeling and prototype demonstration within a limited time, the team used a lightweight Agile-style workflow. Instead of a formal Scrum board, tasks were divided by deliverable area and reviewed frequently through group discussion.

The team divided responsibilities according to the main phases of the System Development Life Cycle: requirements gathering, object-oriented analysis, UML modeling, prototype implementation, report writing, and presentation preparation. This allowed the team to work in parallel while still keeping the final reports and prototype aligned.

1.2 Task Allocation

Team Member	Main Responsibility	Output
Gabriela Caceres	Requirements documentation, report organization, and final report editing	Report structure, requirements text, use case consistency, final proofreading
Juan Azcona	UML modeling and diagram preparation	Use case diagram, sequence diagrams, activity diagrams, class diagram support
Filippo Maldonado	Prototype implementation and technical validation	Electron prototype, model/service implementation, storage and export functions

Sofia Sotelo	Process documentation, interview notes, CRC workshop support, and presentation preparation	Interview notes, decision log, CRC cards, reflection and PPT support
--------------	--	--

The task allocation was flexible. When inconsistencies were found between the prototype, diagrams, and written reports, the team reviewed the affected sections together and revised the design to keep all deliverables consistent.

1.3 Project Management Methodology

The team followed an iterative development process. First, the client interview was used to define the system boundary and main requirements. Second, the requirements were transformed into user stories and use cases. Third, CRC cards and roleplay were used to identify the main classes, responsibilities, and collaborations. Fourth, UML diagrams were produced to formalize the design. Finally, the prototype was implemented and compared with the UML model to detect inconsistencies.

The project was managed through short review cycles. After each major stage, the team checked whether the design still matched the instructor's answers. Important changes, such as removing parallel branching, simplifying versioning, and selecting the Observer Pattern, were recorded in the decision log.

1.4 Timeline

Stage	Activity	Main Output
Day 1	Project brief review and team task division	Initial work plan and interview question blocks
Day 2	First client interview	Raw requirements and initial system scope
Day 3	Requirement clarification and second interview	Refined rules for scenes, validation, dashboard, and versioning
Day 4	CRC workshop and roleplay	Initial class list and responsibility allocation
Day 5	UML modeling	Use case diagram, sequence diagrams, communication diagrams, activity diagrams, and class diagram
Day 6	Prototype implementation and testing	Working desktop prototype with project, scene, validation, compile, search, and storage features
Day 7	Report and presentation preparation	Final Report 1, Report 2, and presentation materials

1.5 Meeting Minutes Summary

Meeting	Main Discussion	Key Decision
Meeting 1	Read project brief and divided interview topics	Four interview blocks were created: users/context, core functions, flows/behavior, and constraints
Meeting 2	Reviewed first interview answers	System confirmed as single-user, offline, desktop-based, and writer-side only
Meeting 3	Clarified ambiguous requirements	Scenes are linear, validation is non-blocking, and unresolved IDs produce warnings
Meeting 4	CRC workshop and object roleplay	Core classes identified: Project, Story, Scene, Character, Location, BackgroundStory, Validator, StoryCompiler, Storage, Dashboard
Meeting 5	UML and prototype consistency review	Observer Pattern selected for dashboard updates; full version history removed from final scope
Meeting 6	Final report review	Reports were revised to align requirements, UML diagrams, design decisions, and prototype behavior

2. Requirements Elicitation Process

2.1 Preparation and Interview Strategy

Before the client interview, we organized the elicitation of the client's vision into four thematic blocks, covering every possible dimension of the system. Questions were grouped into logical sub-blocks; on average, 20 questions were provided in each sub-block, from general context to specific behaviors. In interview sessions one team member would ask the questions and one person would take notes of the instructor's answers which would ensure that no information was missed.

Block	Theme	Focus
Block 1	Users & Context	Roles, permissions, platform, collaboration model, character management
Block 2	Core Functionalities	Scenes, decisions, export formats, dashboard, search, visual graph
Block 3	Flows & Behavior	Session flow, script lifecycle states, branching, error handling
Block 4	Constraints & Edge Cases	Tech stack, storage limits, cycles, deletions, auto-save, crash recovery

2.2 Interview Notes, Key Questions & Answers

2.2.1 First Session

The first session was a full session of all the prepared questions in all four blocks. This session set the framework and direction for the system.

Block	Question	Instructor's Answer
Users & Context	Does the system require an account or login?	Single user no account needed. Focus on features, not authentication.
Users & Context	What technical level is assumed of the user?	No coding required. Non-technical. Just a writer.
Users & Context	Can users have different permission levels?	Writer only make it easy.
Users & Context	Is the editor a desktop or web application?	Desktop application.
Users & Context	Individual or collaborative use?	Individual. Single user, non-collaborator.
Users & Context	One project at a time or multiple in parallel?	Multiple projects in parallel.
Users & Context	Are characters global to the story or branch-specific?	Global. But visibility can be set per chapter/branch.
Users & Context	Can characters have dynamic attributes?	Yes emotional state, personality, trust level, relationships, background story.
Users & Context	Should the system warn if a character is referenced but not created?	Yes.

Users & Context	Writer-side only, or does the reader use the system too?	Writer side only. The system generates a story for reading/printing.
Users & Context	Should the system allow story preview?	Yes a preview/generation mode.
Functionalities	How does the user create a new scene?	New scene button. Each scene is one or more paragraphs in a text box.
Functionalities	Can a scene connect to more than two other scenes?	No. That would repeat the story. Not allowed.
Functionalities	Can two separate scenes converge back into one?	No. Linear only. No parallel branching.
Functionalities	Can the system use dynamic variables in scene text?	Use Character IDs as placeholders; replace with real names at compile time.
Functionalities	What export formats are supported?	MD, PDF, plain text, JSON, minimum is a text file.
Functionalities	Is there a compile/validate function?	Yes. Replaces all IDs with real names. Warns if any ID has no name assigned.
Functionalities	Should the editor display a visual graph?	Linear display. The visual graph is a nice-to-have. Emotional arc graphs are desirable.
Functionalities	Is there a search/filter function?	Yes search by scene and by character, filter by attributes.

Flows & Behavior	What states can a script go through?	Draft → In Progress → Validated → Published. States apply to the generated story, not individual scenes.
Flows & Behavior	Who triggers state transitions?	The user manually.
Flows & Behavior	Can a script go back to a previous state?	Yes, or the user can create a new version/branch manually.
Flows & Behavior	What is the first thing a user does when opening the editor?	Create a scene, edit a character, or edit the background (geography, era, technology).
Constraints	Required programming language or tech stack?	Team's choice no restrictions.
Constraints	Database or local file storage?	Local file storage is fine.
Constraints	Does the system need to run offline?	Yes it's a local desktop app.
Constraints	Is there a maximum number of scenes or characters?	No hard limit up to the writer. Set a large programmatic limit if needed.
Constraints	Should the system auto-save?	Yes, local storage, auto-save always on.
Constraints	What happens if the app crashes?	Auto-save covers recovery, user resumes from the last saved state.

2.2.2. Second session

After reviewing the notes from the first session, we identified some ambiguous or incomplete answers that required further clarification. A second, shorter interview was conducted to resolve these specific points. The questions in this session were more targeted, addressing gaps and edge cases that had emerged during the team's internal analysis.

Block	Question	Instructor's Answer
Scenes	When creating a scene, is there a minimum set of required fields?	No mandatory fields. Leave blank if desired. Validation will report empty attributes as warnings, not blocks.
Scenes	Can a scene belong to more than one story/project?	No. Each scene is exclusive to a single project. Projects are stored separately.
Scenes	Can the same scene appear more than once in the same story flow?	No. If a separate ending is needed, create a separate scene even if 90% is identical.
Scenes	Is there a limit to how many scenes can branch from a decision point?	Maximum 4 branches means creating up to 4 separate stories. No parallel branching within one story.
Scenes	Can a scene exist without being connected to any other scene?	Yes, but validation will flag it as isolated and report it.
Characters	Do character relationships have attributes (e.g., ally/enemy)?	No too complicated to implement.

Characters	If a character is set as not visible, do they still appear in the list?	No, they should not appear.
Characters	Can a character be deleted if already referenced in scenes?	No the system should prevent this.
Background	Can the background story be divided into multiple independent sections?	Yes multiple cards: geography, technology, historical events, environment, etc.
Background	Is the background story global or linked to specific scenes?	Global reference for the entire project.
Background	Can the background story use Character IDs or Scene IDs as references?	Yes.
Export	Does the system replace all IDs automatically on compile?	Yes, automatically. No manual review needed.
Export	If a Character ID has no real name yet, does the system block the export?	No just warn the writer.
Export	What other ID types exist besides character and location?	Character IDs and Location IDs only.

Dashboard	Besides scene/character counts, what else should the dashboard show?	Scenes per character, emotional/action scene counts, scenes with missing IDs, word count per scene.
Dashboard	Can the writer filter search by multiple criteria simultaneously?	Yes filter by attributes (emotion tag, character involved), not by content.
UI/UX	What are the minimal UI/UX requirements?	Edit cards in textboxes. Open and place cards on desktop. No mandatory history section. Type or drag-and-drop scene sequence to generate story.
Versioning	When a new version is created, does the previous remain accessible?	No versioning system too complex. Always the latest version. Version label tag is optional.

2.3 Resulting System Boundary

Based on the complete elicitation process across both sessions, the following scope was established for the Story Scripting Editor:

In Scope

- Single-user desktop application, no authentication required
- Multiple independent Projects (stories), each with its own scenes, characters, and background
- Scene management: create, edit, delete, and sequence scenes within a project
- Character profiles with fixed and user-defined attributes (personality, emotional state, trust level, backstory)

- Background story: multiple cards categorized by geography, technology, historical era, etc.
- Character ID and Location ID system placeholders replaced at compile time
- COMPILE function: validates and replaces all IDs; warns on unresolved entries
- Export formats: Markdown, PDF, plain text, JSON
- Dashboard: scene/character counts, word count, emotion tags, missing ID report
- Search and filter by scene or character attributes
- Story lifecycle states: Draft → Validated → Published. “In Progress” was discussed during elicitation but was consolidated into Draft in the final prototype
- Auto-save to local storage
- Story preview / read mode
- Emotional arc visualization (optional)
- Drag-and-drop or typed scene sequencing (optional if feasible)

Out of Scope

- Multi-user collaboration or conflict resolution
- User authentication or account system
- Web or mobile deployment
- AI-based narrative validation or dead-end detection
- Version history, diff comparison, or rollback
- Conditional branches based on reader choices or runtime flags
- Multiple languages in story content
- Cloud storage or online synchronization
- Parallel branching within a single story

3. The CRC Workshop and Roleplay Execution

3.1 Purpose of the CRC Workshop

After completing the requirements elicitation process, the team conducted a Class–Responsibility–Collaborator workshop to convert the client’s requirements into an object-oriented design. The purpose of the CRC workshop was to identify the main classes of the system, assign clear responsibilities to each class, and determine which classes needed to collaborate during important system operations.

The CRC method helped the team move from a requirement-based view of the system to an object-oriented view. Instead of only asking what the system should do, the team asked which object should be responsible for each behavior. This was especially useful for separating responsibilities between domain classes, service classes, and interface classes.

3.2 CRC Workshop Process

The workshop was conducted after the second interview, when the system boundary was clearer. The team first reviewed the user stories and use cases, then highlighted important nouns and actions from the requirements. Nouns such as project, story, scene, character, location, background, dashboard, validation report, and storage became candidate classes. Actions such as validate, compile, search, save, and update became candidate responsibilities.

The team then created CRC cards for each candidate class. During roleplay, each team member temporarily represented one or more objects. For example, when simulating the “Compile Story” use case, the StoryCompiler object requested the ordered scene list from Story, obtained character and location data from Project, replaced placeholder IDs, and

returned the compiled output. This helped the team confirm that the compiler should be a separate service class instead of placing all compilation logic inside Story.

3.3 CRC Cards

Class	Responsibilities	Collaborators
Project	Stores the project title, stories, characters, locations, and background information; notifies observers when project data changes	Story, Character, Location, BackgroundStory, Dashboard
Story	Maintains an ordered list of scenes; stores lifecycle state; supports scene ordering and story-level status changes	Scene, Project, StoryCompiler, Validator
Scene	Stores scene title, short description, content, emotion tag, action tag, character references, and location references	Story, Character, Location, Validator, StoryCompiler
Character	Stores character ID, name, visibility status, and character attributes such as personality, emotional background, trust level, and backstory	Project, Scene, Validator, StoryCompiler
Location	Stores location ID, name, and descriptive attributes; supports location references in scenes	Project, Scene, Validator, StoryCompiler
BackgroundStory	Stores world-building information for the whole project and organizes it into sections	Project, BackgroundSection

BackgroundSection	Stores one category of background information, such as geography, technology, historical era, or environment	BackgroundStory
Validator	Checks blank fields, unresolved character IDs, unresolved location IDs, and isolated scenes	Project, Story, Scene, Character, Location, ValidationReport
ValidationReport	Stores warnings and errors generated by validation; summarizes validation results for the user	Validator, UI
StoryCompiler	Compiles ordered scenes into a readable story; replaces [CHAR:ID] and [LOC:ID] placeholders with real names	Story, Scene, Project, Character, Location
SearchService	Searches and filters scenes and characters using multiple criteria	Project, Story, Scene, Character
Storage	Saves, loads, and auto-saves project data using local file storage	Project
Dashboard	Displays project metrics such as scene count, character count, word count, tag distributions, and incomplete scenes	Project, Observer
Observable	Maintains the list of observers and notifies them when data changes	Observer, Project
Observer	Defines the update operation that observer classes must implement	Dashboard, Observable

3.4 Roleplay Execution

The team used roleplay to test whether class responsibilities were correctly assigned. In the “Create Scene” scenario, the UI requested a new scene, Scene stored the entered data, Story added the scene to its ordered list, Project notified observers, and Dashboard updated its metrics. This confirmed that dashboard updates should not be directly controlled by Story or Scene.

In the “Validate Project” scenario, Validator inspected the project structure, checked all stories and scenes, identified empty fields and unresolved IDs, and produced a ValidationReport. This roleplay confirmed that validation should be handled by a separate service class rather than by Project itself.

In the “Compile Story” scenario, StoryCompiler requested the ordered scenes from Story, used Project to find character and location names, replaced placeholder IDs, and returned the compiled text. This confirmed that compilation was also a service responsibility and should remain separate from the domain classes.

3.5 Responsibility Shifts After Roleplay

The CRC workshop caused several responsibility changes. First, the team initially considered placing validation logic inside Project, but roleplay showed that this would make Project too large. Therefore, validation was moved to the Validator service class. Second, compilation was also separated into StoryCompiler because compiling scenes and replacing IDs is a processing task, not a basic responsibility of Story. Third, dashboard metric calculation was kept inside Dashboard, while Project only sends update notifications through the Observer Pattern.

As a result, the final design became more modular. Domain classes store and organize project data, service classes process that data, and the dashboard responds to project changes without being tightly coupled to the project model.

3.6 CRC Workshop Evidence

The final CRC cards are documented in Table 3.1. The workshop directly influenced the final class diagram by confirming the main domain classes, separating service responsibilities, and supporting the selection of the Observer Pattern for dashboard updates.

The CRC cards shown in Table 3.1 serve as the finalized reconstruction of the team's CRC workshop results. These cards document the main classes, responsibilities, and collaborators that guided the final UML class diagram and roleplay decisions.

4. Design Evolution & Decision Log

A chronological record of the major architectural decisions made during the design of the Story Scripting Editor, from initial assumptions to final adopted solutions.

Decision 1: Narrative Structure, From Graph to Strictly Linear

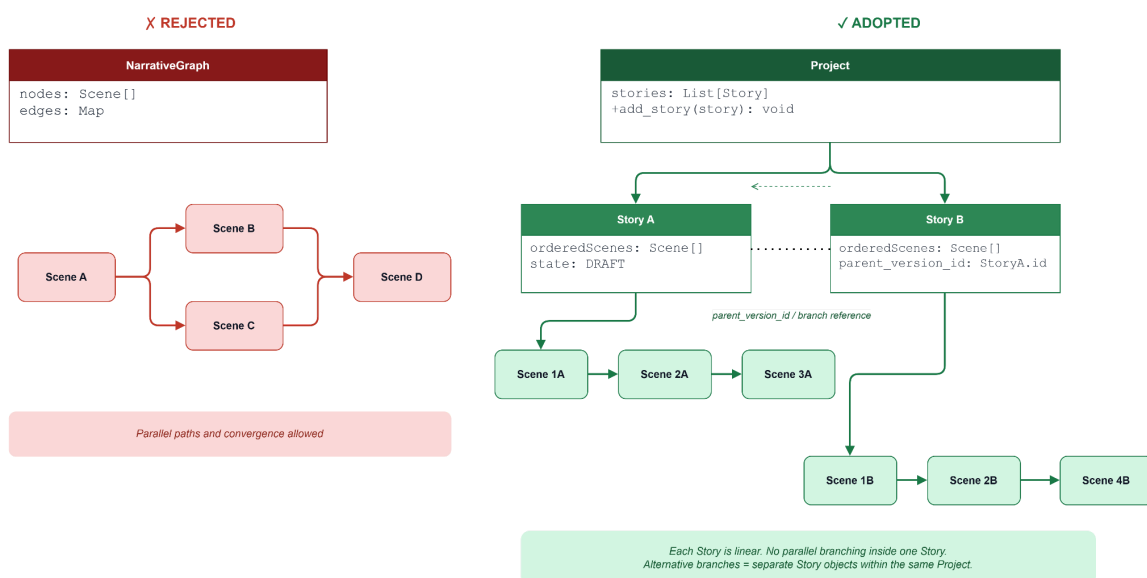
Initial assumption: Scenes could branch freely and converge back, forming a general directed narrative graph.

What changed: The instructor stated: "No, they have to be linear. No parallel branching. 4 branches means 4 separate stories."

Rejected model: NarrativeGraph class with an arbitrary adjacency list.

Adopted model: A Project can contain multiple Story objects, and each Story holds an ordered list of Scene objects. The compiled story follows the scene order defined inside the selected Story. Branching is not modeled as parallel paths inside one Story. If an alternative branch is needed, it is represented as a separate Story within the same Project. This approach preserves a linear structure for each Story while allowing multiple narrative alternatives to coexist in a single Project

Decision 1: Narrative Structure — From Graph to Linear Story Branches



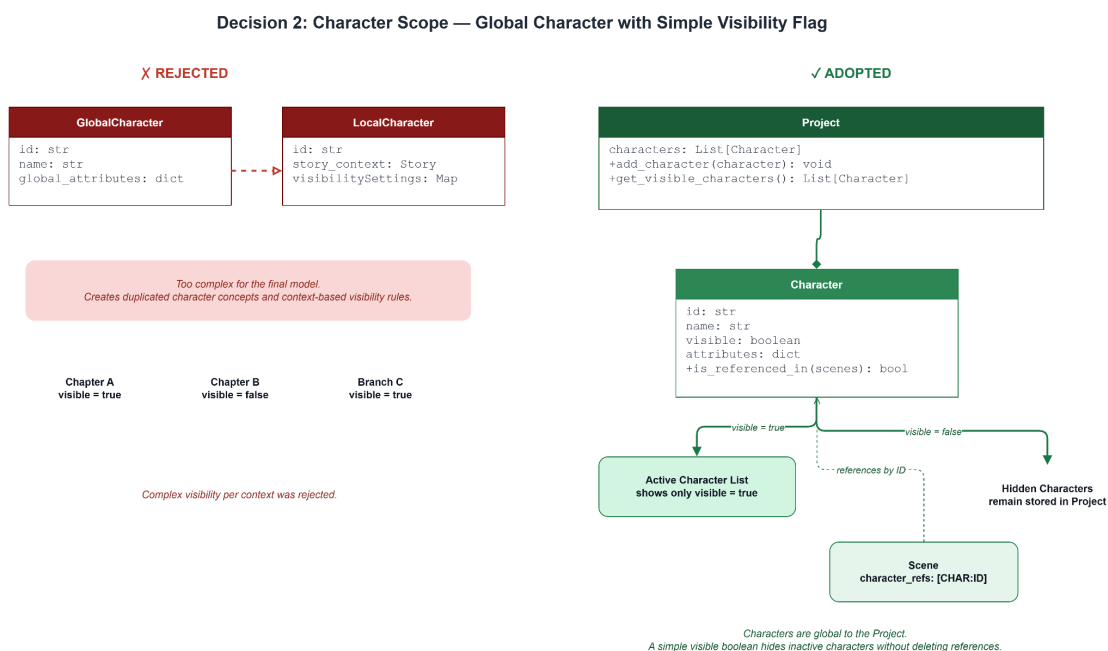
Decision 2: Character Scope: Branch-Specific vs. Global with Visibility

Initial assumption: Two competing models, characters local to each branch, or fully global with no restrictions.

What changed: The instructor clarified that characters should remain global to the Project. During refinement, the team simplified the visibility requirement into a single visible boolean flag. When visible is false, the character is hidden from the active character list, but remains stored in the Project so existing scene references are not lost.

Rejected model: GlobalCharacter and LocalCharacter as separate subclasses, or a complex visibilitySettings map keyed by chapter, branch, or context.

Adopted model: Single Character class with a visible: boolean attribute. Characters set as not visible do not appear in the active character list, but they remain stored in the Project so references are not lost.



Decision 3 Scene Connection Limit: Two-Successor Rule

Initial proposal: Scenes could connect to any number of successors.

What changed: The instructor stated: "Can a scene connect to more than two? No that means you're repeating the story."

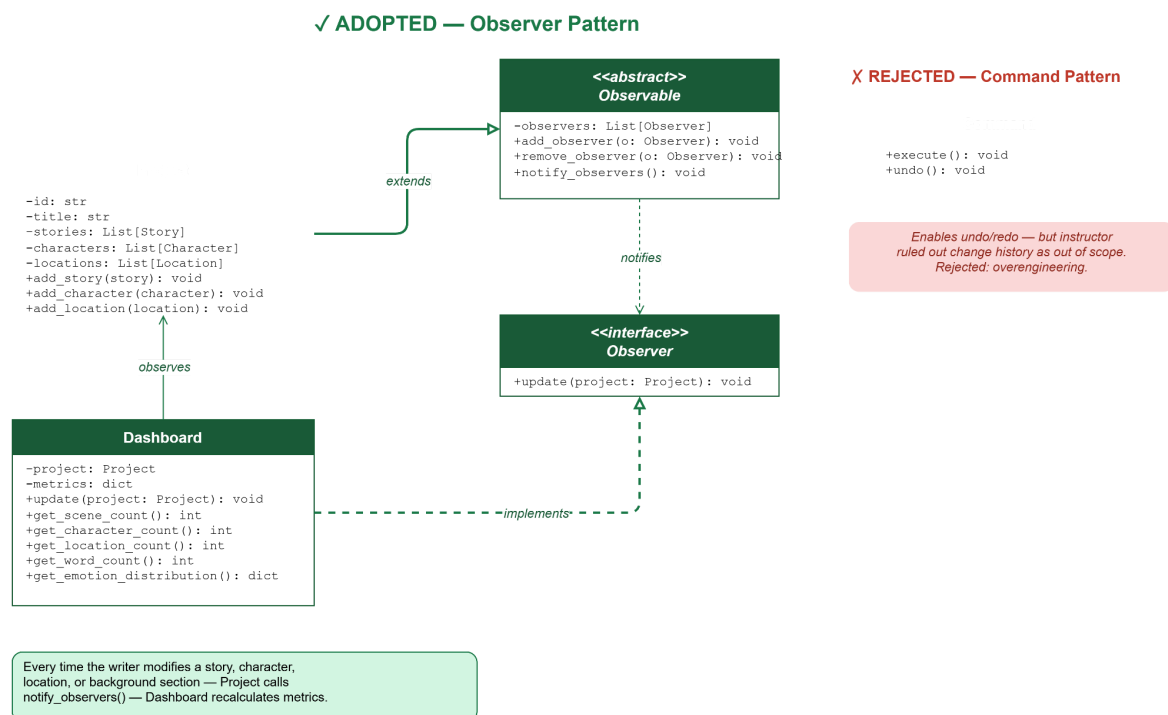
Adopted model: Scene order is managed by the Story object through an ordered scene list. The writer can reorder scenes, but the compiled story follows one defined linear sequence. Isolated scenes are allowed during editing but are flagged by validation as warnings.

Decision 4 Script Lifecycle States: Story-Level, Not Scene-Level

Initial confusion: Should Draft / Validated / Published apply per scene (like Jira cards) or per the generated story?

What changed: The instructor clarified: "This applies to the generated story. The individual cards are always editable - no states on them."

In the final prototype, the lifecycle was simplified to three implemented states: DRAFT, VALIDATED, and PUBLISHED. The “In Progress” state discussed during elicitation was treated as part of the Draft stage because active writing and editing happen before validation.



Decision 5 Design Pattern: State vs. Observer vs. Command

Command: Considered for undo/redo. Rejected - the instructor ruled out changing history as too complex and out of scope.

Observer (selected): Project extends Observable. Dashboard implements Observer. Every time the writer modifies a story, character, location, or background section, Project calls notify_observers() and Dashboard recalculates its metrics. This keeps the two classes loosely coupled.

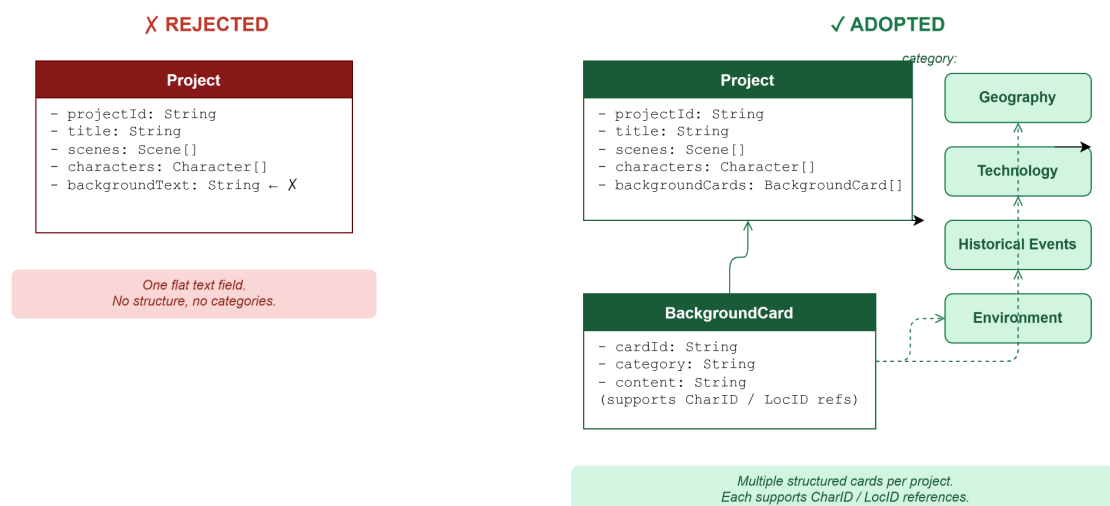
Final decision: Observer as the primary architectural pattern. StoryState is implemented as a simple enum on Story - no separate state classes needed

Decision 6 ID Replacement: Compile-Time Substitution

Initial assumption: Character names typed directly into scene text with no distinction from their display name.

What changed: The instructor introduced placeholder IDs: "Put the character ID in the scene. When you generate the story, replace the ID with the actual name."

Adopted model: StoryCompiler scans scene and background text, resolves Character IDs and Location IDs at compile time, and produces the compiled story output. Unresolved IDs are reported through Validator and ValidationReport. Export remains non-blocking because unresolved IDs produce warnings only.



Decision 7 Background Story: Single Document vs. Multiple Cards

Initial assumption: One free-text background document per project.

What changed: The instructor suggested: "The background can be broken into small boxes: geography, technology, historical events, environment."

Rejected model: backgroundText: String on Project.

Adopted model: BackgroundStory is attached to each Project and contains multiple BackgroundSection objects. Each section represents one category such as geography, technology, historical events, or environment. Background sections can also reference Character IDs and Location IDs.

Decision 8 Validation Philosophy: Non-Blocking Warnings

The question: Should the compiler block export when issues are found (missing IDs, empty fields, isolated scenes)?

Client answer: Non-blocking in every case. "Just warn the writer."

Adopted model: ValidationReport is a pure output artifact with a list of warnings, never a gate. The compiler can still complete the story output even when warnings exist. The Validator produces the ValidationReport, while StoryCompiler focuses on generating the compiled story text.

Decision 9 Versioning: Removed

Initial proposal: Version class that snapshots project state for comparison and rollback.

What changed: The instructor was direct: "No versioning system too complex. Always the latest version."

Rejected classes: Version, VersionHistory, VersionComparator.

Retained: Lightweight Story branching is kept. A new Story can be created with create_version(label), storing parent_version_id and an optional version_label. The prototype does not implement full version history, rollback, comparison, or diff tracking.

5. Individual & Team Reflections

Before we interviewed, we were anticipating the client's required specifications to be organized, and ready to be implemented. Most of the answers given were not completely clear which needed to be redefined in the following session. This was a lesson we had in communicating with clients to understand what they want, and what they actually need.

An important moment that had a profound impact was when the instructor stated that a story must be linear; there cannot be parallel branching in one project. That one clarification changed the entire design of the narrative structure. This was a clear illustration of how one requirement can influence the downstream decisions and how important it is to validate assumptions early.

We were also able to learn and understand that it is as important to know when to go back to the client with a more specific question as it is to ask the question. Overall, this project gave us a firsthand understanding of what requirements elicitation looks like in practice. The experience of moving from ambiguous requirements to concrete client's answers and documented system boundaries was challenging, but it was also where the most meaningful learning happened.

From a technical perspective, one of the main lessons was that object-oriented design is not only about identifying nouns and turning them into classes. The team had to repeatedly evaluate whether each class had a clear responsibility. For example, moving validation and compilation logic into `Validator` and `StoryCompiler` prevented `Project` and `Story` from becoming too large. Similarly, applying the Observer Pattern helped separate dashboard updates from project data management. These decisions made the final system easier to explain, implement, and demonstrate.

The project also showed that diagrams, reports, and prototypes must be kept consistent. Some early design ideas, such as a free narrative graph, full version history, and complex branching, looked useful at first but did not match the clarified client requirements. Revising these ideas helped the team understand that good system analysis is not about adding more features, but about choosing the correct structure for the actual problem.